第四章作业

1 最小覆盖子串

给定字符串 `S`（长度 n）和 `T`（长度 m），请返回 `S` 中能覆盖 `T` 所有字符（每种字符出现次数不少于 `T` 中对应次数）的最短子串及其起始下标（从 0 开始）；若不存在，返回 `("", -1)`。若 `T` 为空，返回 `("", 0)`。

例如 `S = "ADOBECODEBANC"`，`T = "ABC"`，返回 `("BANC", 9)`。

要求：时间复杂度 $O(n+m)$（或 $O(n)$，系数合理即可），可借助 $O(|\Sigma|)$ 额外空间。请说明思路并给出伪代码。

> 原题：力扣 76. 最小覆盖子串（力扣只要求返回子串，本题还要求返回起始下标）。
>
> **T76.最小覆盖子串**
>
> https://leetcode.cn/problems/minimum-window-substring/

```python
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not s or not t or len(t) > len(s):
            return ""
        need = defaultdict(int)
        for ch in t:
            need[ch] += 1
        window = defaultdict(int)
        required = len(need)
        formed = 0
        l = 0
        min_len = float("inf")
        ans_l = 0
        for r in range(len(s)):
            ch = s[r]
            window[ch] += 1
            if ch in need and window[ch] == need[ch]:
                formed += 1
            while formed == required:
                current_len = r - l + 1
                if current_len < min_len:
                    min_len = current_len
                    ans_l = l
                left_ch = s[l]
                window[left_ch] -= 1
                if left_ch in need and window[left_ch] < need[left_ch]:
                    formed -= 1
                l += 1
        if min_len == float("inf"):
            return ""
        return s[ans_l: ans_l + min_len]
```

上述是我在 leetcode 上提交的代码，思路如下：

我期望使用双指针来维护一个滑动的窗口，同时记录窗口内每个字符的出现次数，同时也要记录好目标字符串中每个字符的出现次数。
不断扩大右指针来扩大窗口，每次扩大窗口后，如果窗口内已经满足，那就尝试收缩左指针来达到最小化窗口的目的。
这个想法在检测窗口是否满足时，如果采用朴素的比较会开销过大，所以再维护一个计数器 `formed` 记录当前满足的字符串种类数，当其等于目标字符串中不同字符的种类数时，说明当前窗口已经满足要求。

时间复杂度为 $O(n+m)$，空间复杂度为 $O(|\Sigma|)$，其中 $|\Sigma|$ 是字符集大小。

---

2 KMP 求所有出现位置（允许重叠）

给定模式串 `P`（长度 m）和文本串 `T`（长度 n），`P` 与 `T` 均只含普通字符（无通配符）。请计算并返回 `P` 在 `T` 中**所有**出现的位置的起始下标（从 0 开始，**允许重叠**）。若 `P` 为空，返回空列表。

例如 `P = "abab"`，`T = "abababab"`，返回 `[0, 2]`。

要求：时间复杂度 $O(n+m)$，除结果列表外的额外空间 $O(m)$（用于课件定义的 `next` 数组）。请说明思路并给出伪代码。

> 改编自：力扣 28. 找出字符串中第一个匹配项的下标（力扣只求第一次出现，本题求全部出现且允许重叠）。
>
> **E28.找出字符串中第一个匹配项的下标**
>
> https://leetcode.cn/problems/find-the-index-of-the-first-occurrence-in-a-string/

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> list[int]:
        next = [-1] * len(needle)
        i, k = 0, -1
        res = []
        # 新增 res 记录结果，函数的返回也修改
        while i < len(needle):
            while k >= 0 and needle[i] != needle[k]:
                k = next[k]
            i += 1
            k += 1
            if i == len(needle):
                break
            next[i] = k
            # next[i] = next[k] if needle[i] == needle[k] else k
        i, j = 0, 0
        while j < len(haystack):
        # while i < len(needle) and j < len(haystack):
            if i == -1 or needle[i] == haystack[j]:
                i += 1
                j += 1
            else:
                i = next[i]
```

```
            if i == len(needle):
                res.append(j - len(needle))
                i = next[i]
            # 这个判断为新增
        return res
```

以上是我将 leetcode 上的代码改为求所有出现位置的版本，思路如下：
求出朴素 next 数组，应用 kmp 算法进行匹配，匹配成功后记录下标，并将模式串指针回退到 next 数组对应位置继续匹配。

---

3 利用 next 数组求最短周期

给定字符串 S（长度 n），请利用 KMP next 数组计算 S 的**最短周期**。若 S 为空，返回 0。

若 S 可由某个长度 $p < n$ 的子串重复多次构成（即 $S = X^k$ 且 $|X| = p$），则称 p 为 S 的一个周期，否则记最短周期为 n（即 S 本身）。

例如：

- S = "abcabcabc" → 最短周期 3（"abc" 重复 3 次）
- S = "abcab" → 最短周期 5（无非平凡周期）
- S = "aaaa" → 最短周期 1

要求：时间复杂度 $O(n)$，额外空间 $O(n)$（用于 next 数组）。请说明思路并给出伪代码。

> 对应题：力扣 459. 重复的子字符串（判断是否存在非平凡周期，本题进一步求最短周期）；OpenJudge 02406 字符串乘方（求 $S = a^k$ 的最大 k，即 n/p）。
>
> **E459.重复的子字符串**
>
> https://leetcode.cn/problems/repeated-substring-pattern/
>
> **02406: 字符串乘方**
>
> http://cs101.openjudge.cn/practice/02406/

```
class Solution {
public:
    bool repeatedSubstringPattern(string s) {
        int n = s.length();
        vector<int> pi(n);
        for(int i = 1; i < n; ++i){
            int j = pi[i - 1];
            while(j > 0 && s[i] != s[j]) j = pi[j - 1];
            if(s[i] == s[j]) ++j;
            pi[i] = j;
        }
        int period = n - pi[n - 1];
        return 0 < pi[n - 1] && n % period == 0;
```

```
    }
};
```

总而言之就是求出最长真前后缀长度的数组 pi，然后最短周期如果存在应当为 $n - pi[n-1]$，若 $pi[n-1] > 0$ 且 n 能被最短周期整除，则说明存在周期。

---

4 单次遍历原地删除 'b' 与级联 "ac"

给定字符串 S（长度 n，以字符数组形式存储，可就地修改）。

在一次从左到右扫描中，**原地**（$O(1)$ 额外空间）删除 S 中所有字符 'b'，并删除所有出现的子串 "ac"（删除要**级联**，即删除某个 "ac" 可能使得原本不相邻的 a 与 c 变为相邻，这些新形成的 "ac" 也必须被删除）。要求**单次遍历**（只能从左向右扫一次），最终把保留的字符移到 S 的左端，并返回新长度。

例如：

- "acbac" → ""（全部被删）
- "ababac" → "aa"
- "aaccac" → ""（级联删除）

请给出：**思路说明、伪代码、以及时间复杂度分析**。

> 非力扣原题，是经典面试题（Techie Delight 上有同构题 "Remove all occurrences of AB and C"）。
>
> **E2696.删除子串后的字符串最小长度**（删除 "AB"、"CD"，级联）
>
> https://leetcode.cn/problems/minimum-string-length-after-removing-substrings/
>
> **E1047.删除字符串中的所有相邻重复项**
>
> https://leetcode.cn/problems/remove-all-adjacent-duplicates-in-string/

```
class Solution {
public:
    int minLength(string s) {
        int pos = 0;
        for(char ch : s){
            if(pos > 0 && ((s[pos-1]=='A' && ch == 'B') || (s[pos-1]=='C' && ch ==
'D'))) {
                --pos;
            }else {
                s[pos++] = ch;
            }
        }
        return pos;
    }
};
```

具体思路就是参考 E2696 的解法，使用一个指针 pos 来表示当前保留的字符的末尾位置（相当于用一个指针记录一个栈）
注意如果我们要删除 b 和 ac 的话情况发生变化，我们需要将 if 改为 while 来一直持续判断能不能删除，直到不能删除为止。

```
class Solution:
    def minLength(self, s: str) -> int:
        chars = list(s)
        write = 0
        for c in chars:
            chars[write] = c
            write += 1
            while write > 0:
                if chars[write - 1] == 'b':
                    write -= 1
                elif write >= 2 and chars[write - 2] == 'a' and chars[write - 1]
== 'c':
                    write -= 2
                else:
                    break
        return write
```